

Exercise 8 - Transformers for IceCube2D

Erik Stolt

What you did and how

Similar to *Exercise 4 - Graph Neural Networks* did we implement a network architecture to reconstruct the position (x, y) of the neutrino interaction from the measured Cherenkov light. However, now using transformers, which are a relatively new network architecture with a broad range of applications such as LLMs and image processing tasks. See Appendix 1 for more details on the architecture. More specifically did we implement the transformer in PyTorch and trained for 100 epochs with reduce lr callback. Since this exercise is very similar to exercise 4, just with a different network architecture, did we reuse a lot of code including the code for pre-processing and post-processing.

What results you obtained

An example print-out looked like

Epoch 9/100 | Train Loss: 0.6539 | Val Loss: 0.6885 | lr: 0.0005 | ES: 1 | Time: 25.28s

We also plotted the loss curve

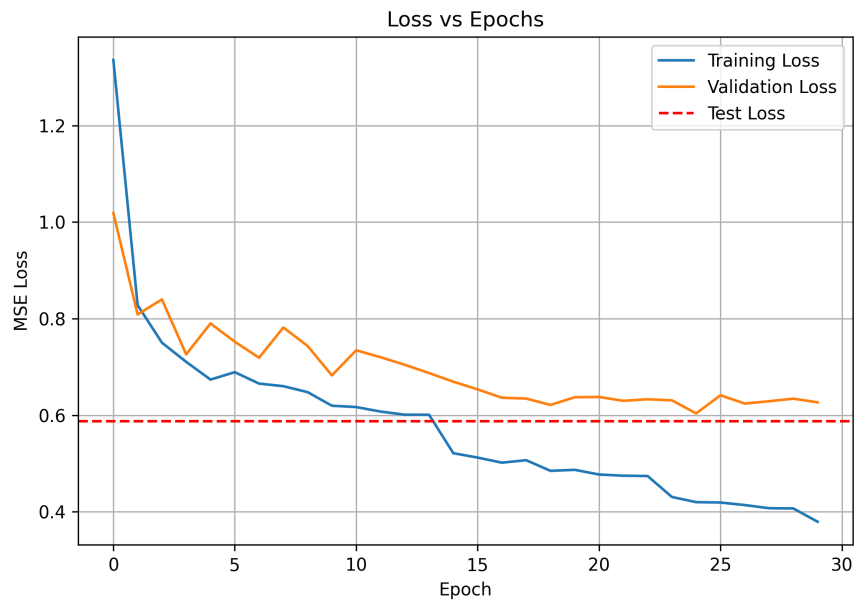


Figure 1: Loss curve for training loss and validation loss as a function of epoch. The red dashed line is the test loss.

as well as a scatter plot showing the predicted vs true positions:

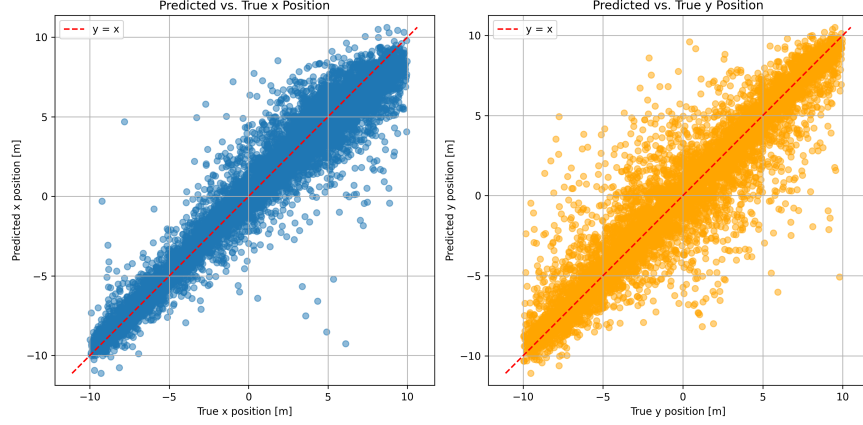


Figure 2: Scatter plot over predicted vs true x and y-positions. Dashed red line shows the ideal predictor.

In order to analyze the accuracy of the model, we also plot a pull distribution, i.e.

$$\text{Pull} = \frac{\hat{y} - y}{\hat{\sigma}}, \quad (1)$$

where $\hat{y}$ is the predicted position and $\hat{\sigma}$ is the std of the numerator in Eq.(1).

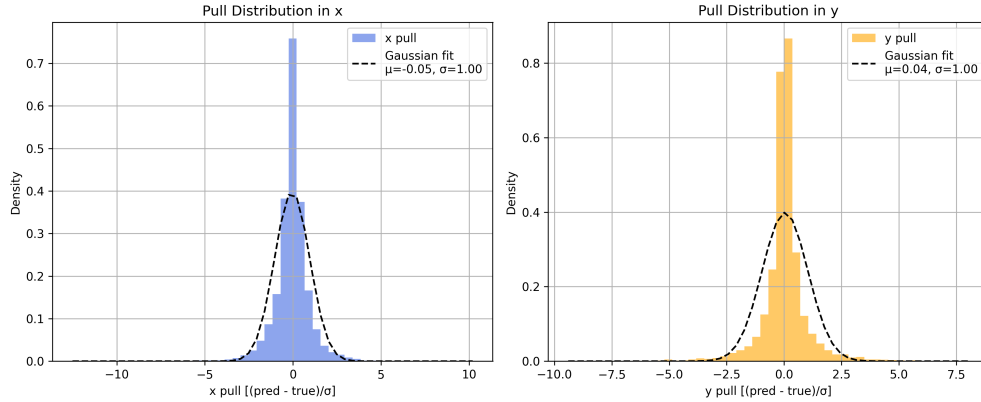


Figure 3: Pull distribution for the positions x and y respectively.

As Fig.3 shows, the means are almost zero and the std is one. This means that the model performs well. I went into detail why in exercise 4, so I will not do it again here.

What challenges you encountered and what could be improved

Since a lot of code was given and we were able to reuse code from exercise 4, there were not many challenges. One minor challenge I encountered was to move the data tensors to device. This was different since in exercise 4 we had a PyTorch tensor. However, in this exercise we had a concatenated list, which means we cannot directly move it to device. We first had to unpack the components, move to device, then repack the list. In theory we replaced

```
data, labels = data.to(device), labels.to(device)
```

with

```

data_vec, lengths = data      # unpack from list
data_vec = data_vec.to(device) # move tensor to GPU
labels = labels.to(device)    # move labels to GPU
data = [data_vec, lengths]    # repackage data.

```

Link to GitHub

Link: <https://github.com/erst6955/Advanced-Applied-Deep-Learning-in-Physics-And-Engineering>

Appendix 1

Transformers work by, given an input sequence $X \in \mathbb{R}^{N \times D}$, each token is projected into *queries*, *keys*, and *values*:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}, \quad \mathbf{V} = \mathbf{X}\mathbf{Q}^{(v)}, \quad \mathbf{W}^{(q)}, \mathbf{W}^{(k)}, \mathbf{W}^{(v)} \in \mathbb{R}^{D \times D}. \quad (2)$$

The single attention head computes

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{Softmax} \left[\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D}} \right] \mathbf{V}, \quad (3)$$

instead of

$$\mathbf{Y}(\mathbf{X}) = \text{Softmax}[\mathbf{X}\mathbf{X}^\top] \mathbf{X}, \quad (4)$$

in order to introduce learnable parameters and non-linearity. Furthermore, one divides by $\sqrt{D}$ for stability reasons. To enhance capacity, *multi-head attention* uses H parallel attention heads. In particular, we let $\mathbf{H} \rightarrow \mathbf{H}_h$ which mean that we have

$$\begin{aligned} \mathbf{Q}_h &= \mathbf{X}\mathbf{W}_h^{(q)}, \\ \mathbf{H}_h &= \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \quad \text{with} \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(k)}, \\ &\quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(v)}. \end{aligned} \quad (5)$$

Then we concatenate the $\mathbf{H}_i$ and multiply with another learnable matrix $\mathbf{W}^{(o)}$ according to

$$\mathbf{Y}(\mathbf{X}) = \text{Concat}[\mathbf{H}_1, \dots, \mathbf{H}_H] \mathbf{W}^{(o)} \quad (6)$$

where each head independently applies attention with its own parameters. In fact, a transformer layers consists of:

$$\begin{aligned} \mathbf{H}_h &= \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h), \\ \mathbf{Y}(\mathbf{X}) &= \text{Concat}[\mathbf{H}_1, \dots, \mathbf{H}_H] \mathbf{W}^{(o)}, \\ \mathbf{Z} &= \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}], \\ \tilde{\mathbf{X}} &= \text{LayerNorm}[\text{MLP}[\mathbf{Z}] + \mathbf{Z}], \end{aligned} \quad (7)$$

which we compactly denote $\tilde{\mathbf{X}} = \text{TransformerLayer}[\tilde{\mathbf{X}}]$. Note that we in multiple stages have the concatenation of previous input similar to RNNs structures. This is to provide a stable gradient and preserve the non-abstract representation of the input data. However, there is still one missing component. The transformer to this

end does not have any perspective of the order of the input which is crucial for many applications such as LLMs. To implement this one simply adds (not concatenate since addition works and is more computationally efficient) a positional encoding vector to the input:

$$\tilde{\mathbf{x}} = \mathbf{x}_n + \mathbf{r}_n, \quad (8)$$

where, similar to binary encoding:

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even,} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd.} \end{cases} \quad (9)$$

This encoding meets the requirements we wish to have:

- Unique representation for each position
- Bounded
- Generalize to longer sequences
- Relative distance between two positions independent of absolute position

This architecture allows the model to capture both local and global dependencies with high parallelism.